



UNIVERSITY OF
WOLVERHAMPTON



HERALD
COLLEGE
KATHMANDU

Artificial Intelligence and Machine Learning Task-2 Report

Name: Anup Giri
Student ID: 2417555
Tutor: Durga Pokhrel
Date: 2026/05/10

Contents

Abstract	3
1. Introduction.....	3
2. Dataset	3
3. Methodology	4
3.1 Data Preprocessing and Augmentation.....	4
3.2 Part A — CNN Architectures from Scratch	4
3.2.1 Baseline CNN.....	4
3.2.2 Deeper CNN with Regularization.....	5
3.3 Part B — Transfer Learning with MobileNetV2	6
4. Experiments and Results	6
4.1 Baseline vs. Deeper Architecture	6
4.2 Computational Efficiency	6
4.3 Optimizer Analysis: Adam vs. SGD	7
4.4 Ablation Study	7
4.5 Transfer Learning Performance	7
5. Conclusion and Future Work.....	8

Abstract

This report is a comprehensive image classification project for the image classification dataset, based on the whole deep learning pipeline, ranging from data acquisition, data preprocessing steps, designing the model, training it, evaluating it, and transferring it. This project is divided into 2 parts: Part A – Design and train Convolutional Neural Networks (CNNs) from scratch and Part B – Transfer Learning for CNN with the MobileNetV2 Architecture pre-trained on ImageNet. This data set consists of labelled images from various categories of scenes (buildings, forests, glaciers, mountains, seas, streets). A three convolutional layer, three dense layer baseline CNN was trained and tested, which was followed by an architecture with Batch Normalisation and Dropout to obtain a deeper regularised architecture. We did comparative analysis with different models, different optimisers (Adam vs. SGD), and an ablation study to understand how removing Dropout affects the model. The transfer learning was done in two steps: extracting features from MobileNetV2 using frozen convolutional layers, and fine-tuning the top layers. The results demonstrate that the transfer learning approach can substantially surpass the two scratch-trained architectures in terms of classification accuracy, and also impose significantly reduced training time as it exploits pre-learned feature representations. All the training was performed with Google Colab with GPU support.

1. Introduction

Image classification is an important task in computer vision and it has a wide range of applications such as autonomous navigation, medical imaging, remote sensing, and content moderation. This project focuses on scene classification, which is a problem of assigning a semantic category label to an image, according to its overall visual content. This is difficult as the same visual element may appear in different scenes and the same class may have high intra-class variation.

Convolutional Neural Networks are the most widely used architecture for image recognition owing to the ability to learn hierarchical spatial features by sharing weights in the convolution and pooling layers. The project looks into two aspects of CNN: building CNNs from scratch and transfer learning where high-capacity CNNs can be pre-trained on large scale datasets like ImageNet and then fine-tuned to new, small scale datasets with much lower computational cost.

The goals of this project are to train and evaluate a baseline CNN model, a deeper regularised CNN, and a fine-tuned MobileNetV2 model on the scene classification dataset; to compare the performance of various optimisers (e.g., Adam, RMSProp, and SGD); to perform the analysis of the ablative model and to conclude the pros and cons of architecture complexity, training time and classification accuracy.

2. Dataset

The dataset used in this project is a scene classification dataset organized into class subdirectories for training and testing. The dataset includes images of natural and urban scenes, structured for multi-class image classification using Keras ImageDataGenerators.

Attribute	Details
Task	Multi-class Scene Image Classification
Input Format	RGB images (JPEG)
Baseline Image Size	128 x 128 pixels
Transfer Learning Image Size	224 x 224 pixels
Train/Validation Split	80% training, 20% validation (via ImageDataGenerator)
Data Augmentation	Rotation (20 degrees), horizontal flip
Normalisation	Pixel rescaling to [0, 1] by dividing by 255

Prior to model training, a dataset cleaning step was performed using PIL's image verification method. Each image file was opened and verified to detect corruption. Only images that passed verification were copied to a clean destination directory. This process was applied independently to both training and test partitions, ensuring data integrity throughout the pipeline.

Exploratory data analysis revealed the class distribution via bar charts and sample image grids, providing an understanding of the dataset's balance and visual diversity. Image dimensions were inspected to confirm consistent sizing after resizing by the data generator.

3. Methodology

3.1 Data Preprocessing and Augmentation

All images were preprocessed using Keras ImageDataGenerators. For training data, pixel values were rescaled to [0, 1], and augmentation was applied including random rotations up to 20 degrees and horizontal flipping. These augmentation techniques increase the effective diversity of the training set, improving model generalization and reducing overfitting. Validation and test generators applied only rescaling, without augmentation, to preserve the integrity of evaluation metrics.

3.2 Part A — CNN Architectures from Scratch

3.2.1 Baseline CNN

The baseline model was built following the specification of three convolutional layers followed by three fully connected layers. Each convolutional block consists of a Conv2D layer with ReLU activation followed by a MaxPooling2D layer, progressively doubling the number of filters ($32 \rightarrow 64 \rightarrow 128$) to capture increasingly abstract features. A Flatten layer transitions from spatial to

vector representation. Three Dense layers (256, 128, 64 units) with ReLU activations precede the softmax output layer.

Layer	Type	Parameters
1	Conv2D (32 filters, 3x3, ReLU) + MaxPool	Spatial feature extraction
2	Conv2D (64 filters, 3x3, ReLU) + MaxPool	Richer feature maps
3	Conv2D (128 filters, 3x3, ReLU) + MaxPool	High-level features
4	Flatten	Vectorise feature maps
5	Dense (256, ReLU)	Fully connected classification
6	Dense (128, ReLU)	Intermediate representation
7	Dense (64, ReLU)	Fine-grained abstraction
Output	Dense (num_classes, Softmax)	Class probability distribution

3.2.2 Deeper CNN with Regularization

The deeper model more than doubles the baseline's convolutional depth by introducing paired convolutional blocks before each pooling layer. Batch Normalization was added after the first convolution in each pair to stabilize activations and accelerate convergence. Dropout layers (rates of 0.5 and 0.3) were introduced after the two largest Dense layers to reduce overfitting by randomly deactivating neurons during training.

Layer Group	Components	Purpose
Block 1	Conv2D (32) + BN + Conv2D (32) + MaxPool	Low-level edge and texture features
Block 2	Conv2D (64) + BN + Conv2D (64) + MaxPool	Mid-level pattern features
Block 3	Conv2D (128) + BN + Conv2D (128) + MaxPool	High-level semantic features
Dense Layers	Dense(512, Dropout 0.5) + Dense(256, Dropout 0.3) + Dense(128)	Classification head
Output	Dense(num_classes, Softmax)	Final class distribution

3.3 Part B — Transfer Learning with MobileNetV2

MobileNetV2, pre-trained on ImageNet, was selected for transfer learning. This model is highly efficient due to its inverted residual and linear bottleneck design, making it suitable for datasets where computational resources are limited. Images were resized to 224x224 pixels to match the expected input dimensions.

Transfer learning was applied in two stages. In the feature extraction stage, all MobileNetV2 convolutional layers were frozen, and a new classification head (GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.5) → Dense(num_classes, Softmax)) was trained from scratch for 5 epochs. In the fine-tuning stage, layers beyond index 100 in the base model were unfrozen, and the full model was retrained for a further 5 epochs using a very small learning rate (1e-5) to make subtle adjustments to the pre-trained weights without catastrophic forgetting.

4. Experiments and Results

4.1 Baseline vs. Deeper Architecture

Both models were trained for 10 epochs using the Adam optimizer with categorical cross-entropy loss. Training and validation loss curves were plotted for comparison. The deeper model, benefiting from Batch Normalization and Dropout, demonstrated more stable training and generally improved validation accuracy compared to the baseline. Adding more convolutional layers and filters allowed the model to learn richer feature hierarchies, resulting in better discrimination between visually similar scene categories.

The training loss of the baseline model decreased faster in early epochs due to its simpler architecture, but the deeper model's regularization techniques helped prevent overfitting as reflected in its more consistent validation metrics.

4.2 Computational Efficiency

The deeper model required longer training time per epoch than the baseline due to its greater number of parameters and the additional Batch Normalization computations. However, GPU acceleration in Google Colab substantially mitigated this overhead. Training times were recorded for all models, and the deeper model's computational cost was justified by its improved accuracy. Transfer learning demonstrated the most favourable accuracy-to-training-time ratio, achieving high accuracy in just 10 total epochs (5 feature extraction + 5 fine-tuning).

Model	Epochs	Image Size	Key Characteristic	Training Time
Baseline CNN	10	128x128	3 Conv + 3 Dense, no regularization	Fastest (scratch)
Deeper CNN	10	128x128	6 Conv + BN + Dropout	Moderate
Transfer Learning (MobileNetV2)	5 + 5	224x224	Frozen base + fine-tuning	Shortest total

4.3 Optimizer Analysis: Adam vs. SGD

A simplified comparison model was trained independently using both SGD (learning rate 0.01) and Adam (learning rate 0.001) for 5 epochs. Training accuracy curves were plotted for both. Adam consistently achieved faster convergence and higher accuracy within the limited epoch budget, owing to its adaptive learning rate mechanism which adjusts per-parameter step sizes based on gradient history. SGD converged more slowly, particularly in early epochs, though it may reach comparable performance with longer training and learning rate scheduling.

4.4 Ablation Study

An ablation study was conducted by removing the Dropout layer from a simplified CNN. Training and validation accuracy curves were compared against the regularized models. Without Dropout, the model exhibited a wider gap between training and validation accuracy, indicating higher overfitting. This confirms that Dropout is an effective regularization strategy for this dataset, preventing the model from memorizing training-specific patterns and forcing it to learn more generalizable features.

4.5 Transfer Learning Performance

The fine-tuned MobileNetV2 model outperformed both scratch-trained CNNs on test accuracy. This result is expected given that MobileNetV2's convolutional base, trained on over a million ImageNet images, has already learned rich multi-scale feature representations for natural and artificial objects. Feature extraction (frozen base) yielded strong initial accuracy, and fine-tuning the upper layers further improved performance by adapting the pre-learned representations to the specific visual characteristics of the scene classification dataset.

Predictions on test images were compared across the baseline, deeper, and transfer learning models, demonstrating that the MobileNetV2-based classifier produced more confident and accurate classifications, particularly for visually similar categories such as mountains and glaciers, or buildings and streets.

5. Conclusion and Future Work

This project successfully implemented and compared three deep learning approaches for scene image classification: a baseline CNN from scratch, a deeper regularized CNN, and a fine-tuned MobileNetV2 transfer learning model. The results demonstrate a clear performance hierarchy: transfer learning significantly outperforms scratch-trained models in both accuracy and training efficiency, confirming its practical value for datasets of moderate size.

The ablation study confirmed the importance of Dropout for preventing overfitting, and the optimizer comparison showed that Adam is superior to SGD for rapid convergence on image classification tasks. Batch Normalization contributed to training stability in the deeper model.

Key limitations include the fixed 128x128 image size for scratch models (larger resolutions may capture finer details) and the 10-epoch training ceiling, which may not allow models to reach their full potential. Future work could explore higher-capacity transfer learning backbones such as ResNet50 or EfficientNet, hyperparameter search for optimal learning rates and batch sizes, and test-time augmentation to improve inference robustness.